



Xác định vị trí đối tượng (Object Localization) Phần 2

Dẫn nhập



Phân loại

Dog

Hộp giới hạn

Xác định
vị trí đối
tượng



Nội dung

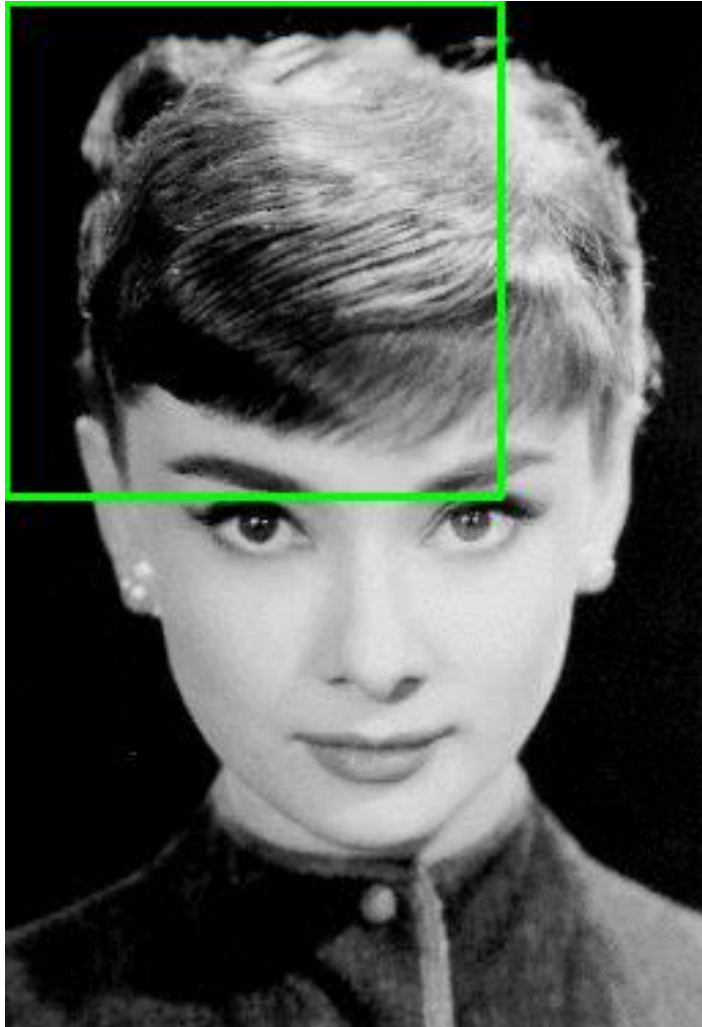
- Thuật toán NMS
- Phương pháp sử dụng mạng học sâu



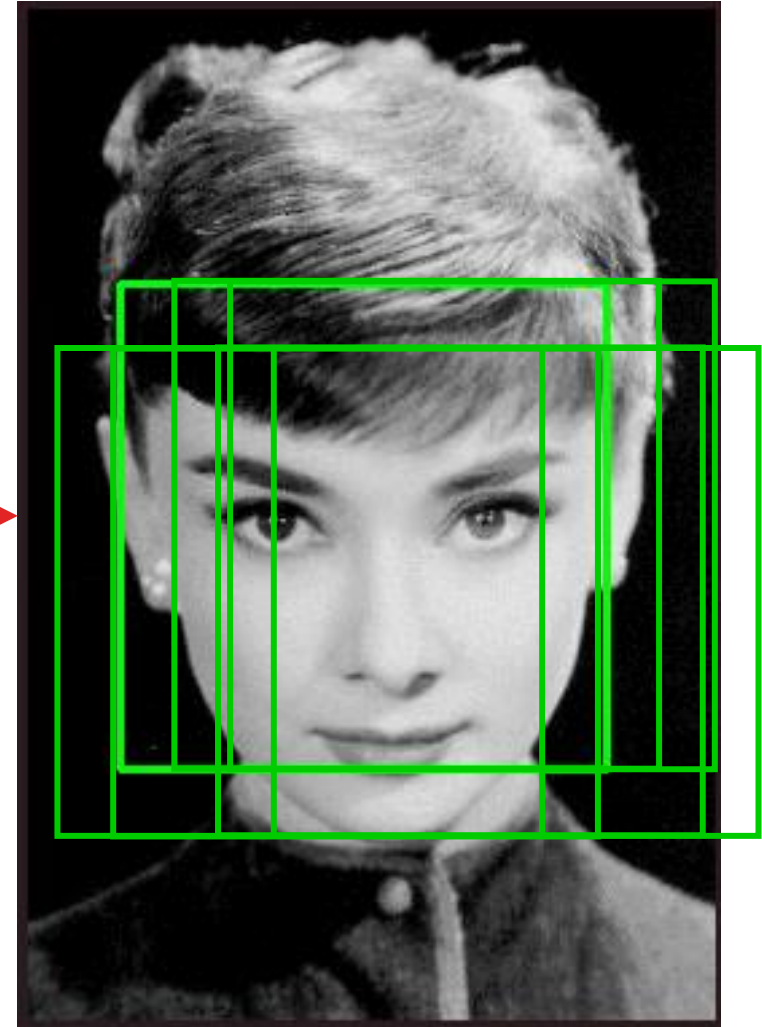
Thuật toán NMS

Non-Maximum Suppression

Giới thiệu

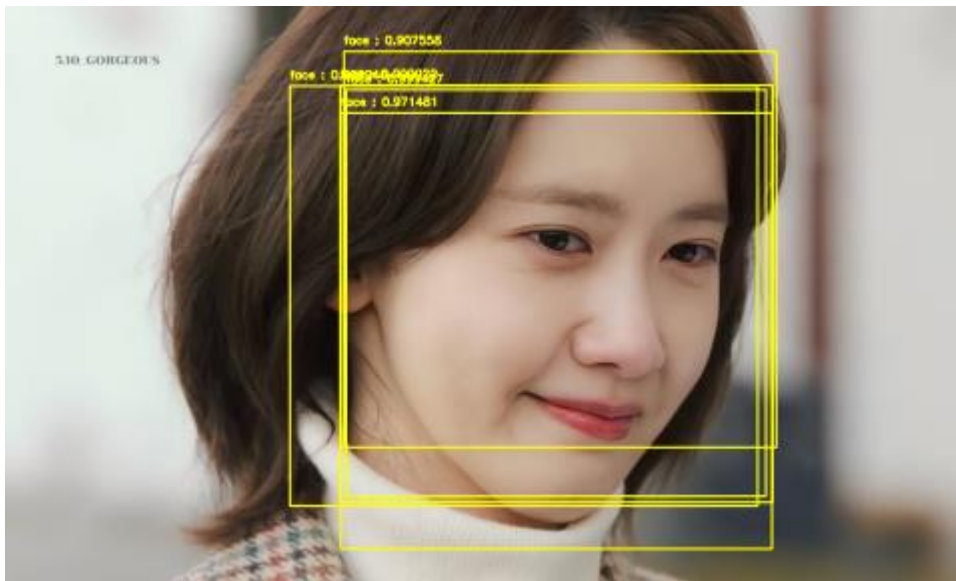


Sliding Windows +
Classifiers

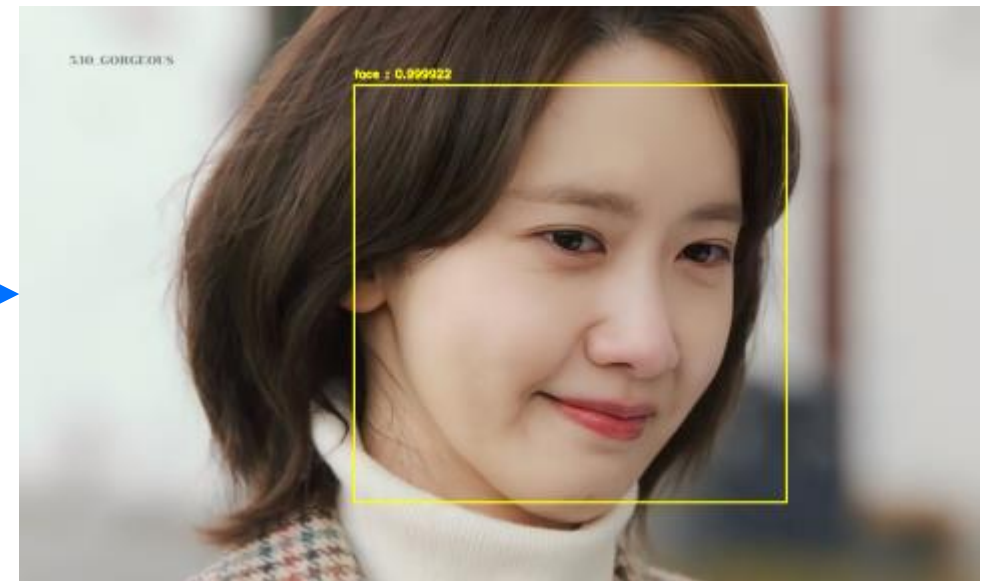


Giới thiệu

- Giúp loại bỏ các hộp giới hạn dư thừa



NMS



Nguồn: <https://viblo.asia/p/tim-hieu-va-trien-khai-thuat-toan-non-maximum-suppression-bJzKmr66Z9N>



Mô tả thuật toán

Algorithm 1 Non-Max Suppression

```
1: procedure NMS( $B, c$ )
2:    $B_{nms} \leftarrow \emptyset$    Initialize empty set
3:   for  $b_i \in B$  do  $\Rightarrow$  Iterate over all the boxes
4:      $discard \leftarrow \text{False}$    Take boolean variable and set it as false. This variable indicates whether b(i)
                                   should be kept or discarded
5:     for  $b_j \in B$  do   Start another loop to compare with b(i)
6:       if  $\text{same}(b_i, b_j) > \lambda_{nms}$  then   If both boxes having same IOU
7:         if  $\text{score}(c, b_j) > \text{score}(c, b_i)$  then
8:            $discard \leftarrow \text{True}$    Compare the scores. If score of b(i) is less than that
                                   of b(j), b(i) should be discarded, so set the flag to
                                   True.
9:         if not  $discard$  then   Once b(i) is compared with all other boxes and still the
                                   discarded flag is False, then b(i) should be considered. So
10:           $B_{nms} \leftarrow B_{nms} \cup b_i$    add it to the final list.
11:   return  $B_{nms}$    Do the same procedure for remaining boxes and return the final list
```

Nguồn: <https://towardsdatascience.com/non-maximum-suppression-nms-93ce178e177c>

Mô tả thuật toán

- **Input:**

- Một tập các hộp giới hạn: $P = \{(x_1, y_1, x_2, y_2, c)\}$, trong đó:
 - (x_1, y_1, x_2, y_2) lần lượt là tọa độ điểm top-left và bottom-right của bounding box
 - c là confidence score tương ứng với box đó.
- Giá trị ngưỡng IoU để loại bỏ các bounding box thừa: **thresh_iou**

- **Output:**

- Một mảng **keep** chứa các bounding box sau khi đã được loại bỏ đi các bounding box dư thừa.

<https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/>

Mô tả thuật toán

- **Bước 1:**

Chọn box $S \in P$ có confidence score cao nhất trong tập P , loại bỏ box đó ra khỏi tập P và thêm box đó vào tập **keep**.

- **Bước 2:**

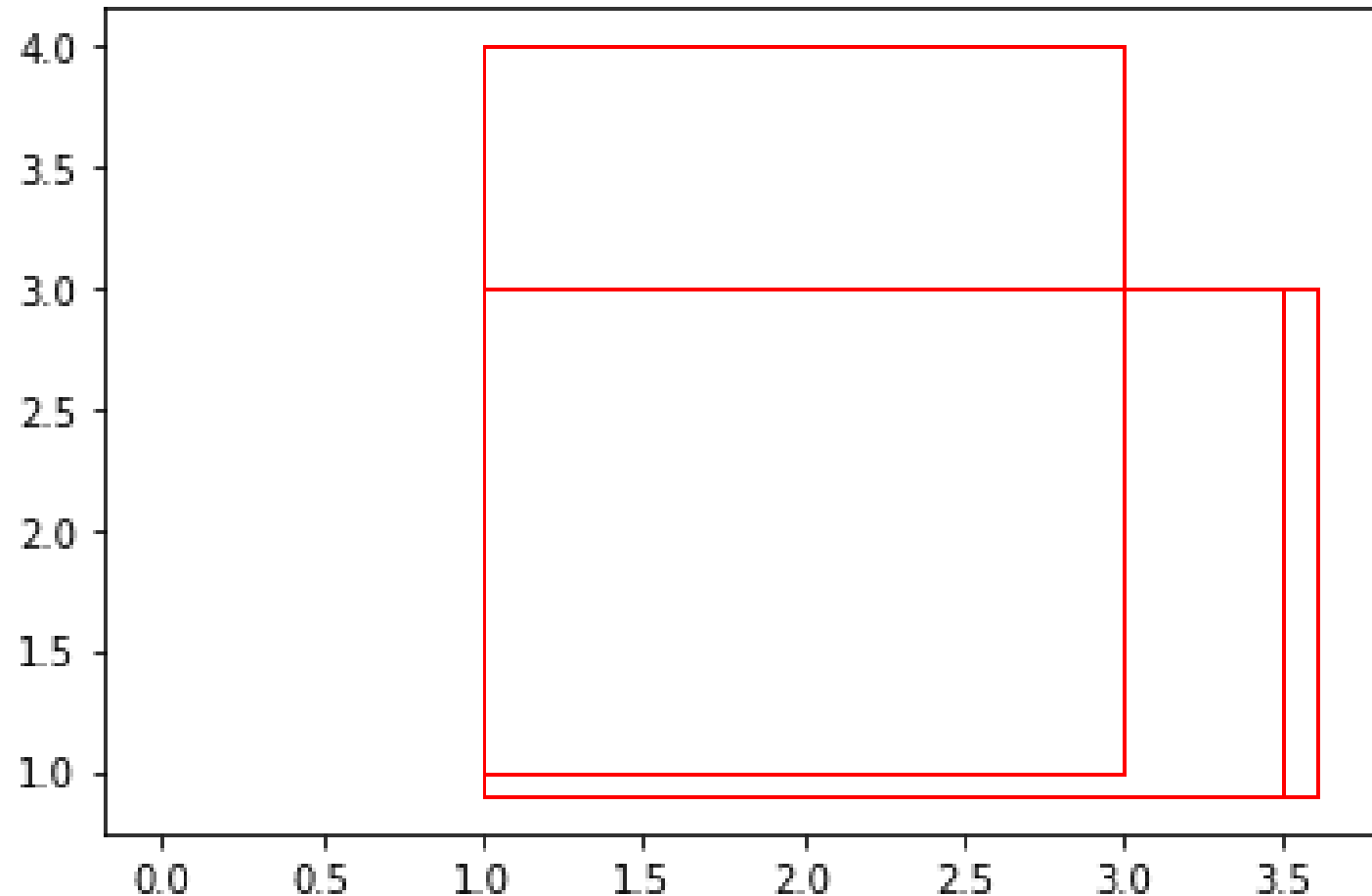
- Thực hiện **tính IoU** giữa box S vừa lấy ra ở bước 1 với toàn bộ các box còn lại trong tập P .
- Nếu có box nào trong P có IoU với box S đang xét mà lớn hơn ngưỡng **thresh_iou** thì loại bỏ box đó ra khỏi P .

- **Bước 3:**

Lặp lại bước 1 cho đến khi P không còn box nào.

Ví dụ 1:

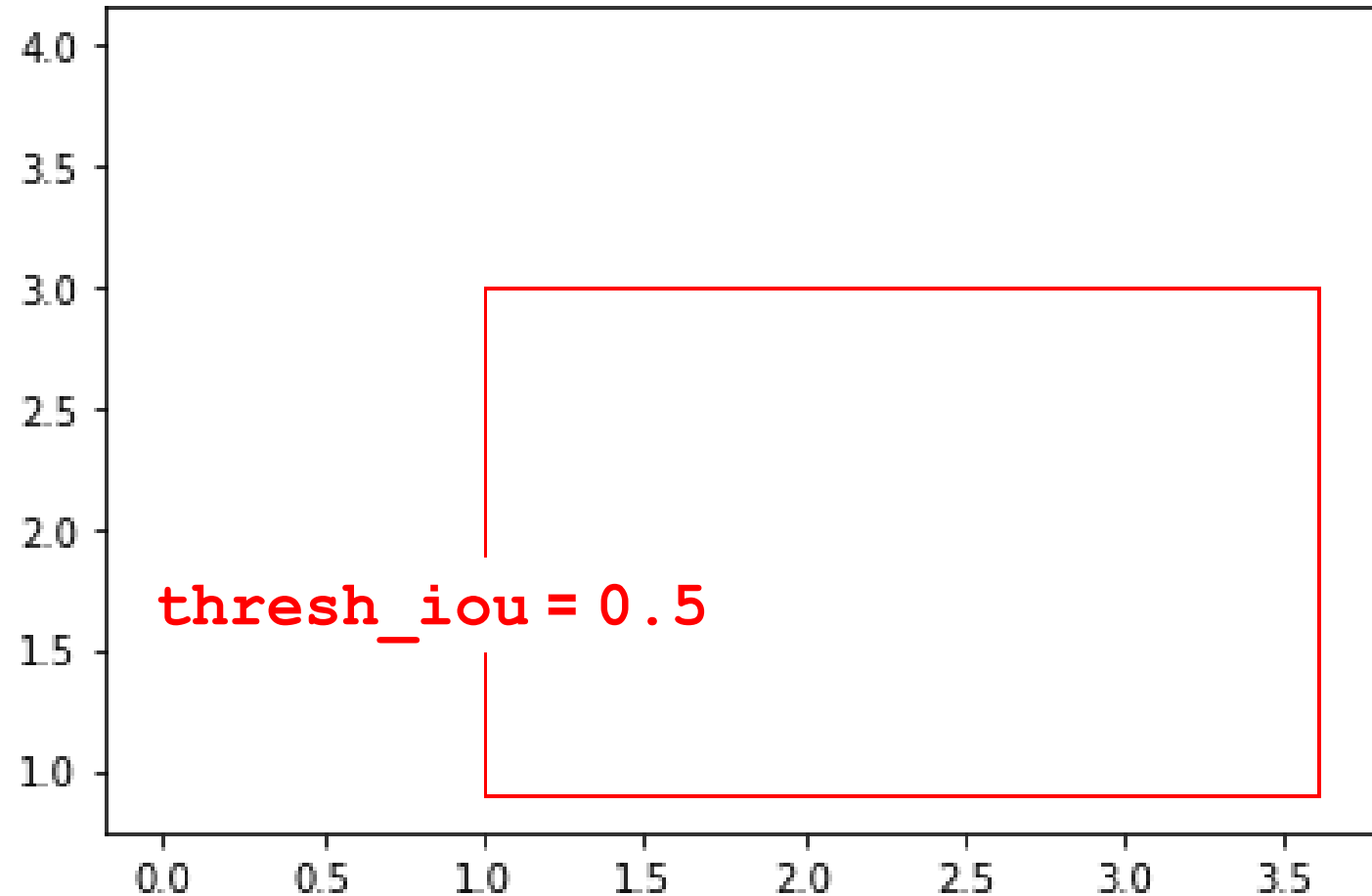
```
P = torch.tensor([
    [1, 1, 3, 3, 0.95],
    [1, 1, 3, 4, 0.93],
    [1, 0.9, 3.6, 3, 0.98],
    [1, 0.9, 3.5, 3, 0.97]
])
```



Nguồn: <https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/>

Ví dụ 1:

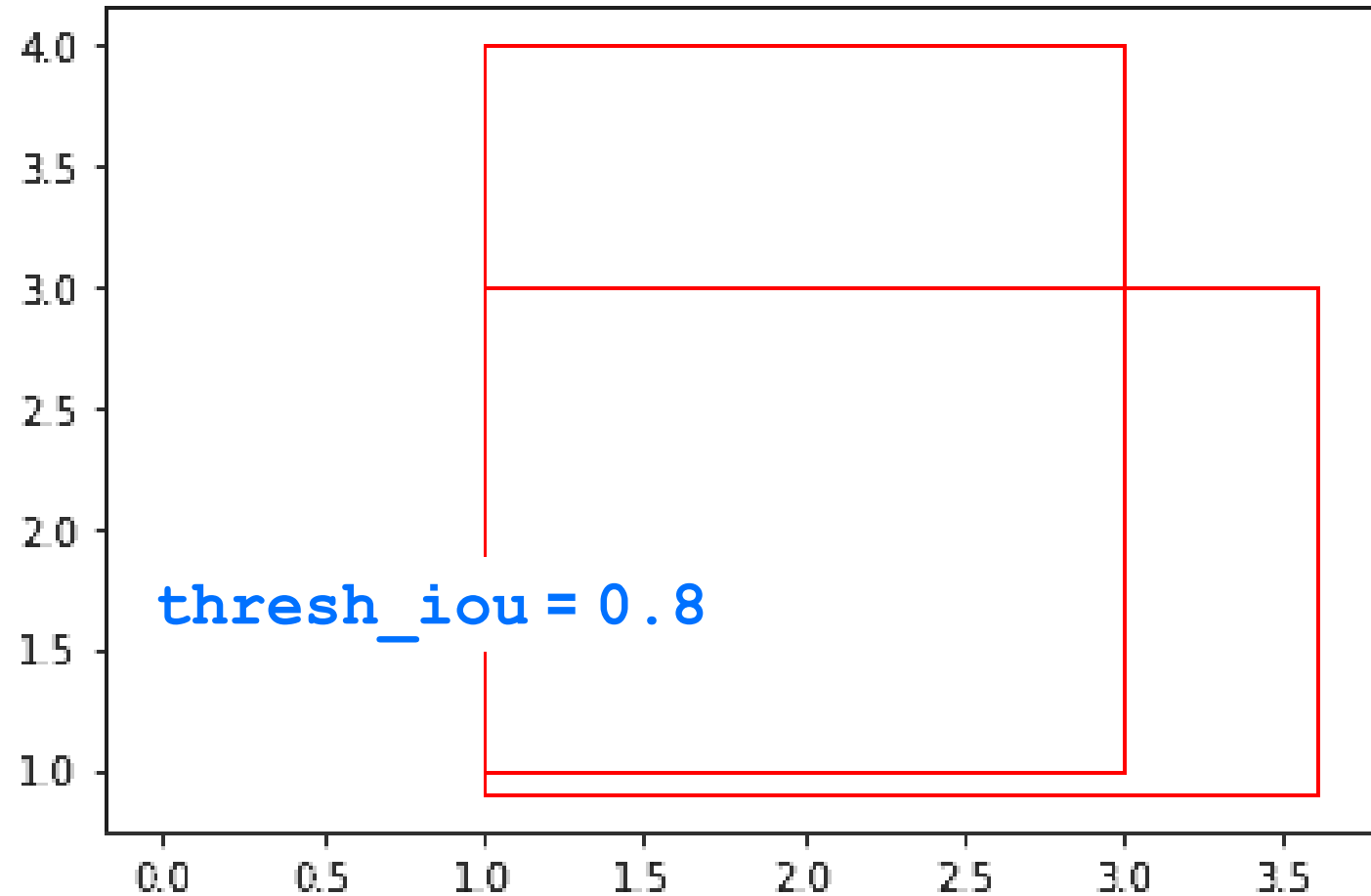
```
P = torch.tensor([
    [1, 1, 3, 3, 0.95],
    [1, 1, 3, 4, 0.93],
    [1, 0.9, 3.6, 3, 0.98],
    [1, 0.9, 3.5, 3, 0.97]
])
```



Nguồn: <https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/>

Ví dụ 1:

```
P = torch.tensor([
    [1, 1, 3, 3, 0.95],
    [1, 1, 3, 4, 0.93],
    [1, 0.9, 3.6, 3, 0.98],
    [1, 0.9, 3.5, 3, 0.97]
])
```



Nguồn: <https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/>

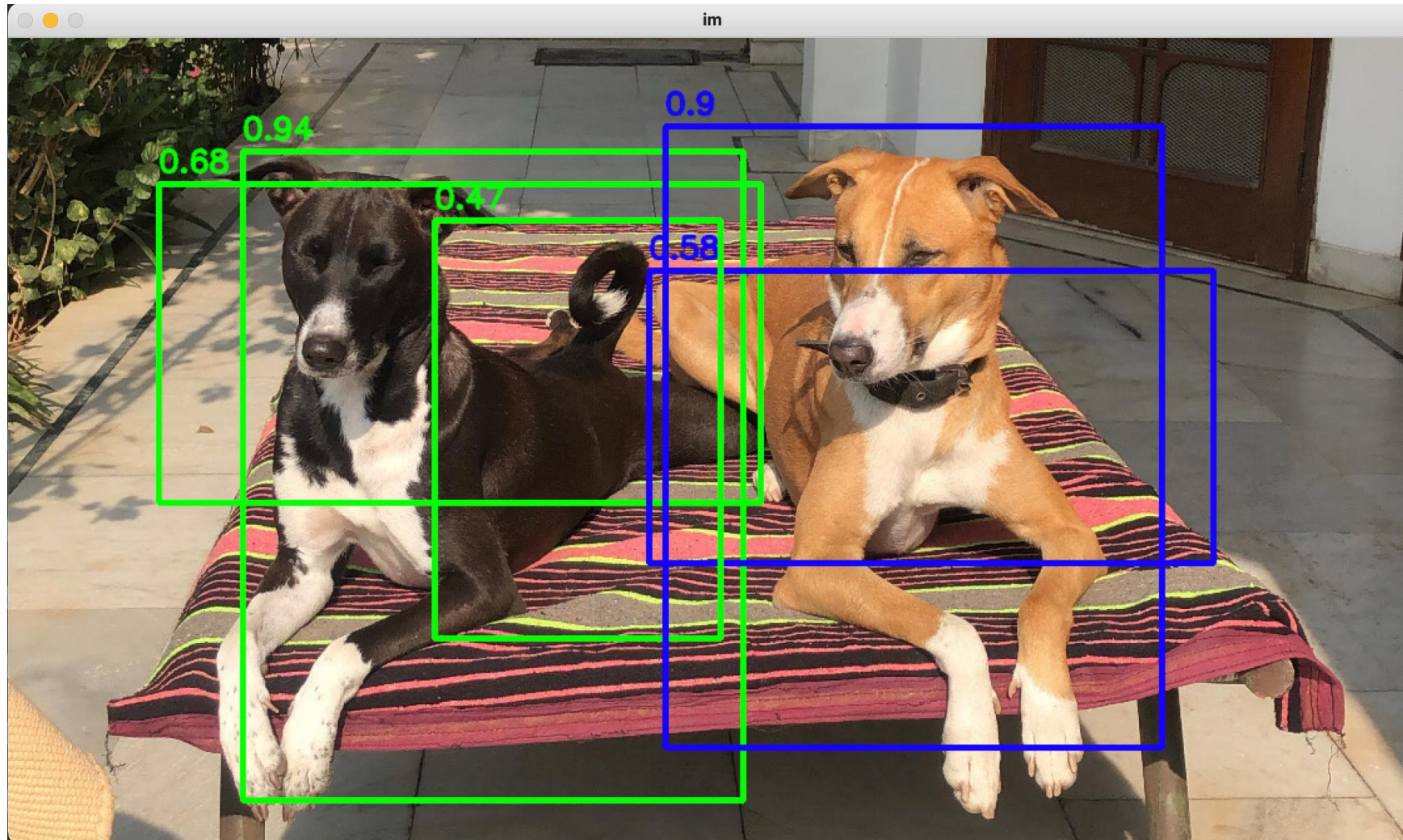


Ví dụ 2: Áp dụng cho nhiều đối tượng

Danh sách tọa độ của các đối tượng

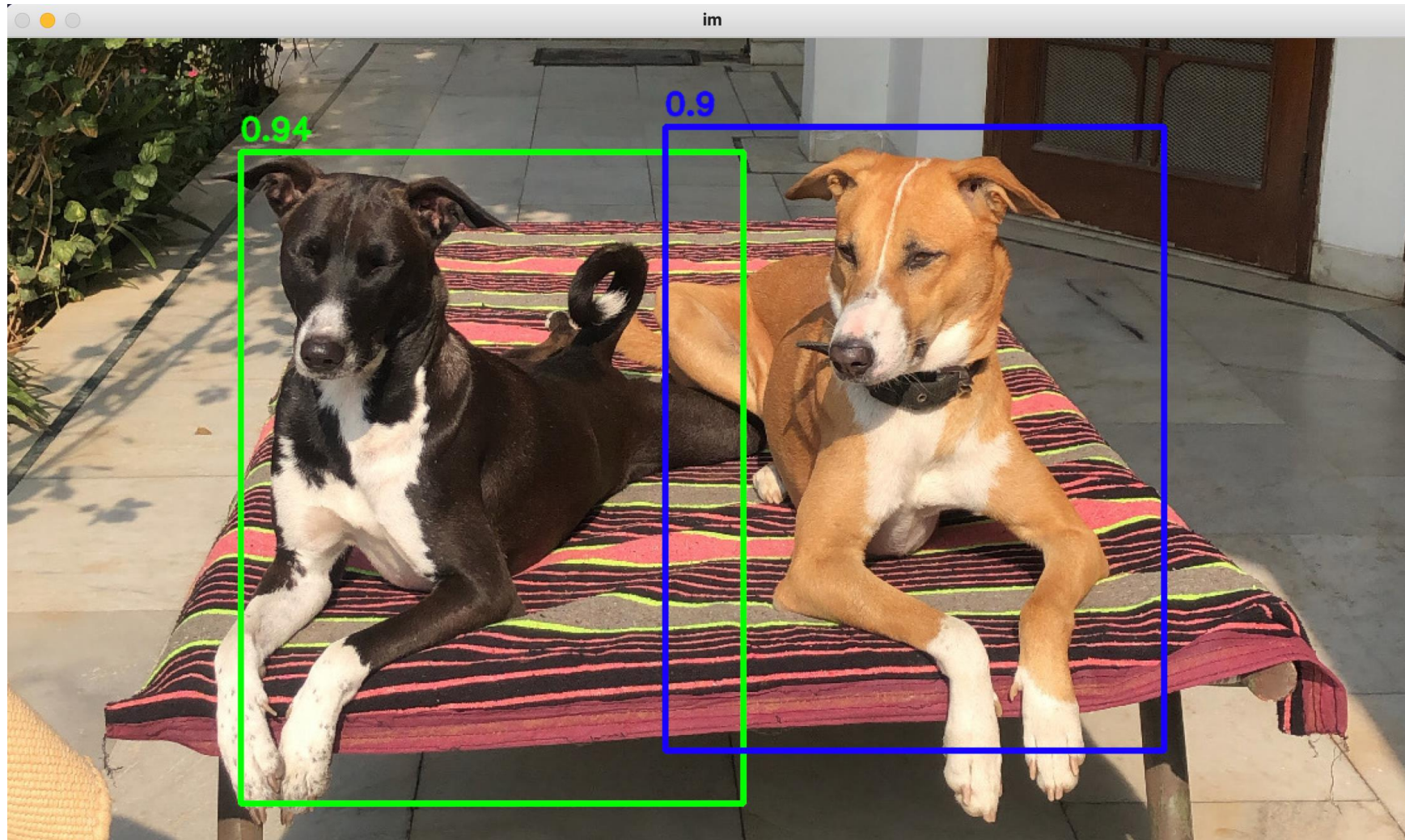
- 0 187 90 586 607 0.94
- 0 120 116 600 370 0.68
- 1 511 185 961 418 0.58
- 0 340 145 568 478 0.47
- 1 524 70 920 565 0.92

Ví dụ 2: Áp dụng cho nhiều đối tượng



<https://towardsdatascience.com/intuition-and-implementation-of-non-max-suppression-algorithm-in-object-detection-d68ba938b630/>

Ví dụ 2: Áp dụng cho nhiều đối tượng



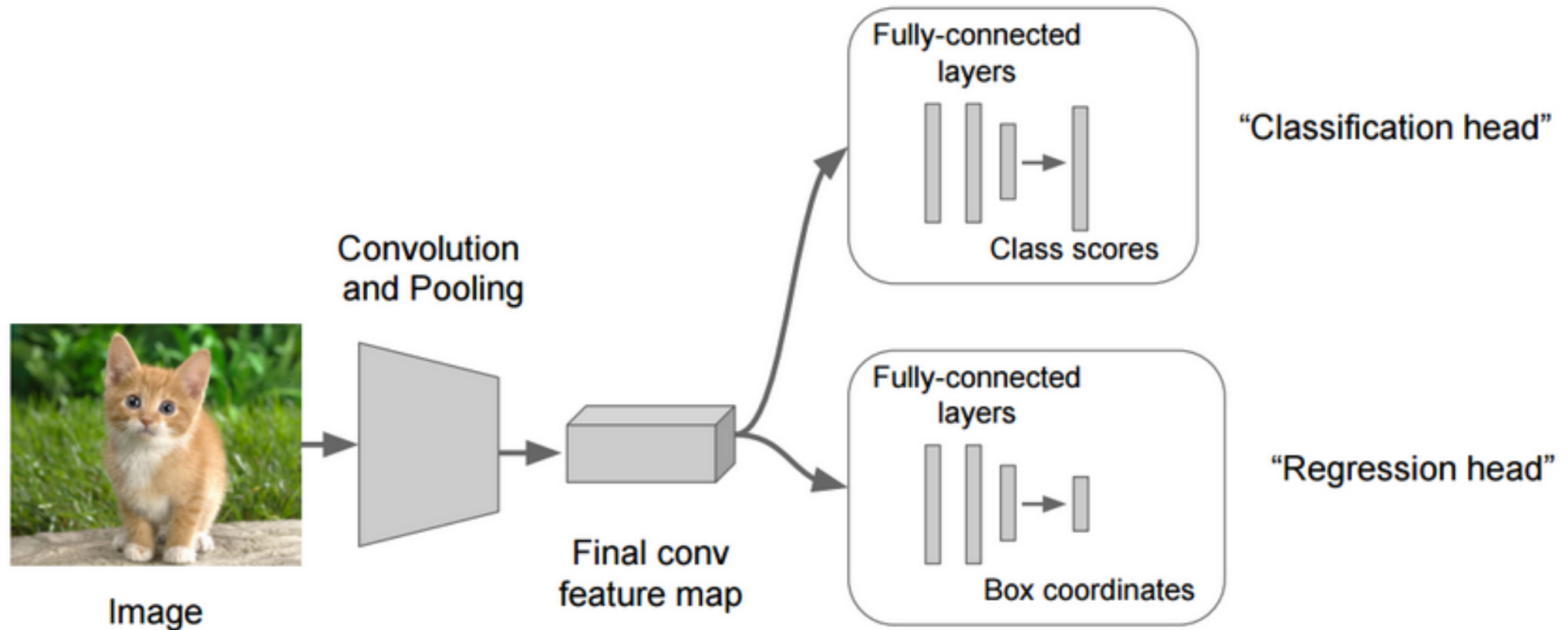
<https://towardsdatascience.com/intuition-and-implementation-of-non-max-suppression-algorithm-in-object-detection-d68ba938b630/>

Thực hiện bởi Trường Đại học Công nghệ Thông tin, ĐHQG-HCM



Phương pháp sử dụng mạng học sâu

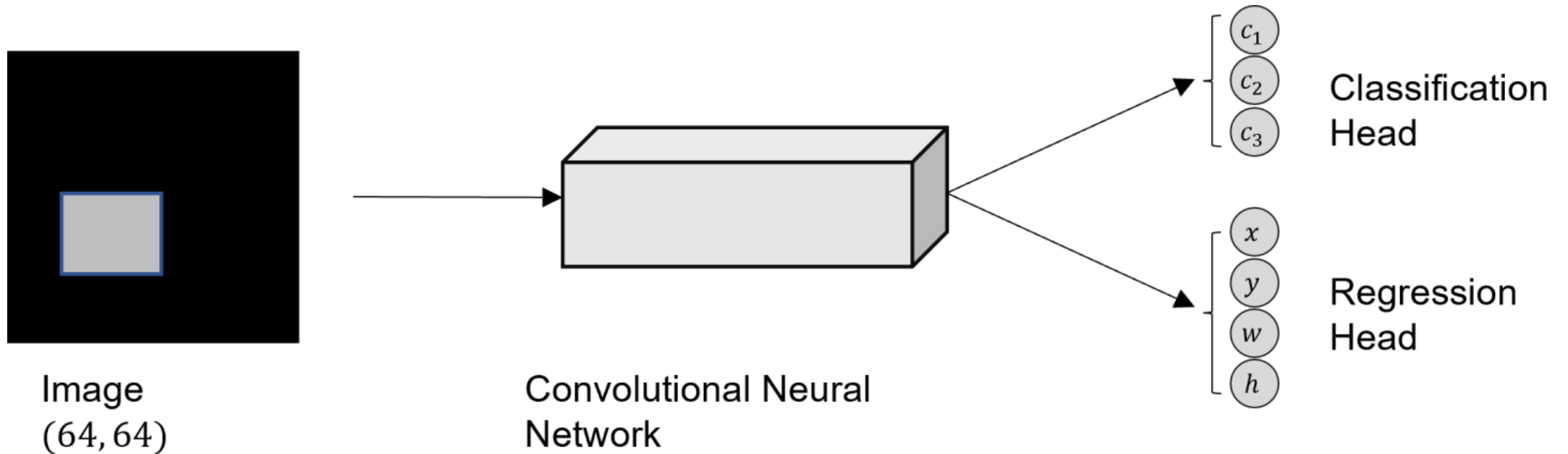
Phương pháp sử dụng mạng học sâu



https://leonardoaraujosantos.gitbook.io/artificial-intelligence/machine_learning/deep_learning/object_localization_and_detection

Phương pháp sử dụng mạng học sâu

- Mô hình cơ bản





Phương pháp sử dụng mạng học sâu

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense
from tensorflow.keras.layers import Flatten, Dropout
from tensorflow.keras import Input, Model
```

```
inputs = Input(shape=(image_size, image_size, 1))
x = Conv2D(16, kernel_size=(3,3), activation="relu")(inputs)
x = MaxPooling2D(pool_size=(2,2))(x)
x = Conv2D(32, kernel_size=(3,3), activation="relu")(x)
x = MaxPooling2D(pool_size=(2,2))(x)
x = Flatten()(x)
x = Dropout(0.5)(x)
```

```
class_out = Dense(3, activation="softmax", name="class")(x)
bbox_out = Dense(4, activation="sigmoid", name="bbox")(x)
```

```
model = Model(inputs=inputs, outputs=(class_out, bbox_out))
```

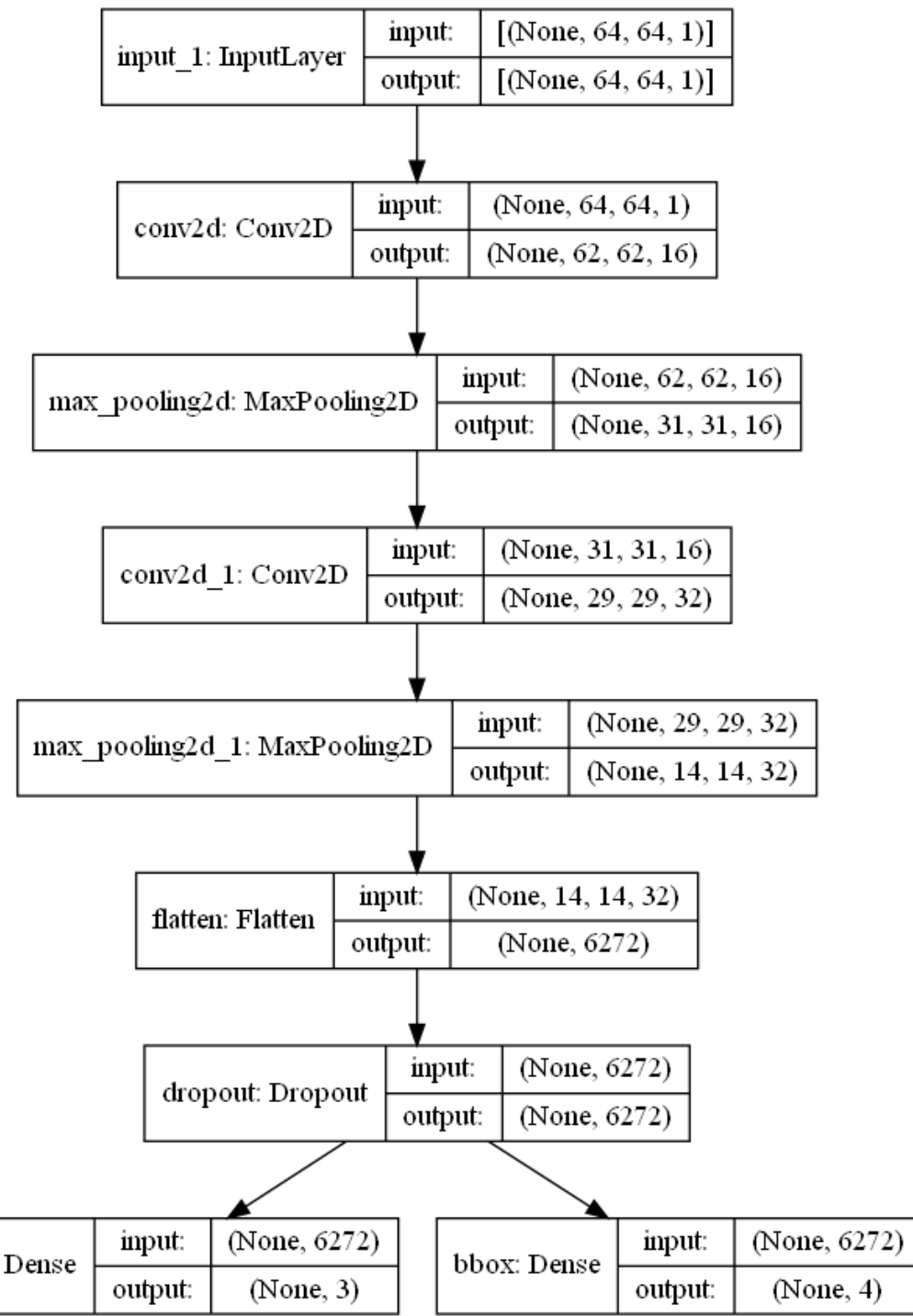
Phương pháp sử dụng m

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.layers import Input, Model
```

```
inputs = Input(shape=(image_size, image_size, 3))
x = Conv2D(16, kernel_size=(3, 3), activation='relu')(x)
x = MaxPooling2D(pool_size=(2, 2))(x)
x = Conv2D(32, kernel_size=(3, 3), activation='relu')(x)
x = MaxPooling2D(pool_size=(2, 2))(x)
x = Flatten()(x)
x = Dropout(0.5)(x)
```

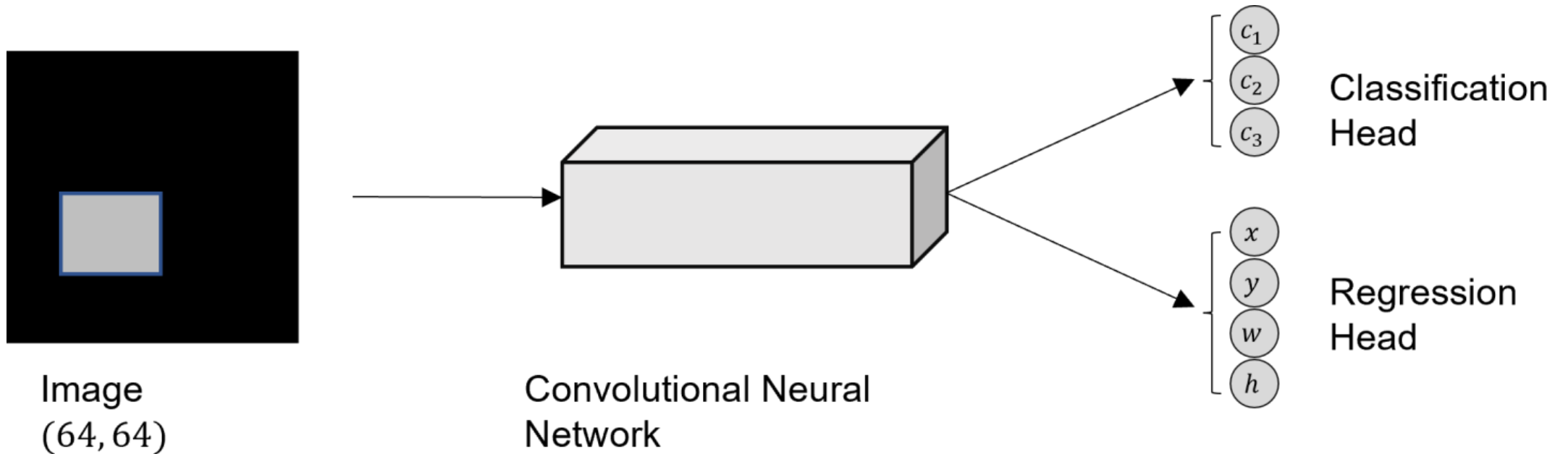
```
class_out = Dense(3, activation='softmax')(x)
bbox_out = Dense(4, activation='sigmoid')(x)
```

```
model = Model(inputs=inputs, outputs=(class_out, bbox_out))
```



Phương pháp sử dụng mạng học sâu

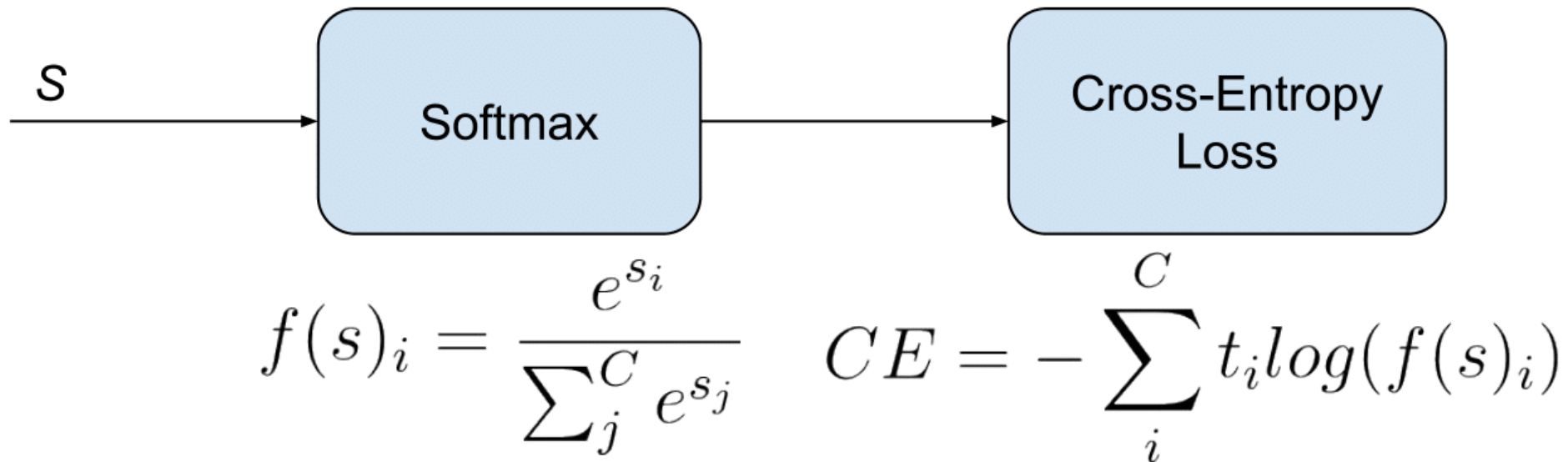
- Mô hình cơ bản



```
model.compile("adam", loss=['categorical_crossentropy', 'mse'],  
              metrics=["accuracy"])
```

Phương pháp sử dụng mạng học sâu

- Đánh giá bài toán phân loại



https://gombru.github.io/2018/05/23/cross_entropy_loss/

Phương pháp sử dụng mạng học sâu

- Đánh giá bài toán Hồi quy để tìm Hộp giới hạn

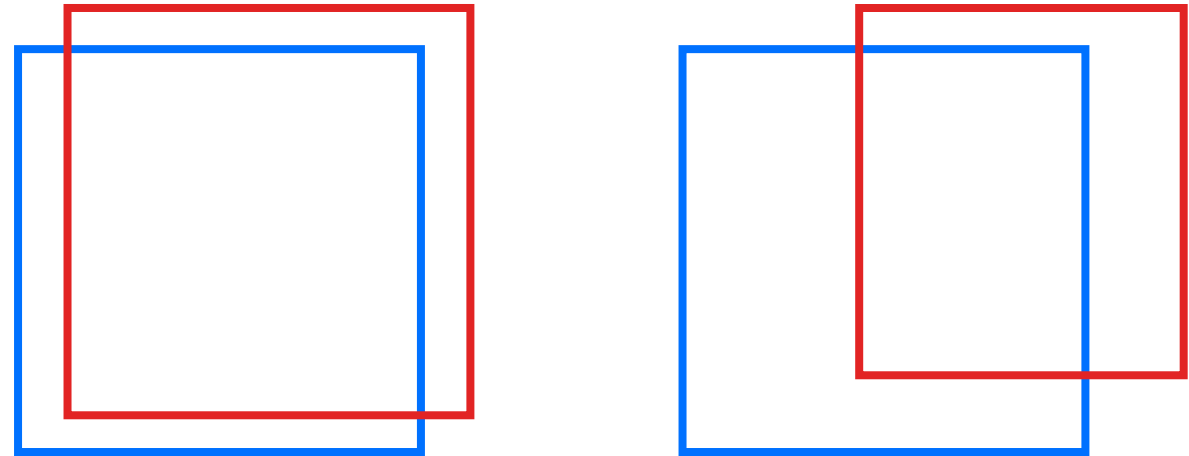
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values





Ví dụ: Sử dụng mạng ResNet50

```
class MyNetwork:
    def __init__(self):
        self.model=None
    def build(self):

        inputs = Input(shape=input_shape)
        base = ResNet50(include_top=False, weights='imagenet', input_tensor = inputs)

        x = base.output
        x = GlobalAveragePooling2D()(x)
        x = Dense(1024, activation='relu')(x)

        classification_output = Dense(num_classes, activation='softmax', name='label')(x)
        localization_output = Dense(4, activation='sigmoid', name='bounding_box')(x)

        self.model = Model(inputs=inputs, outputs=(classification_output, localization_output))
```



Ví dụ: Sử dụng mạng ResNet50

```
def train(self, x_train, y_train, x_val, y_val, learning_rate, epochs):
    opt = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    self.model.compile(
        optimizer=opt,
        loss=['categorical_crossentropy', 'mse']
    )

    file_path="/kaggle/working/checkpoint.model.keras"
    callbacks = [
        ModelCheckpoint(file_path, verbose=1, save_best_only=True, mode='auto'),
        ReduceLROnPlateau(monitor='val_loss', factor=0.1, patience=5, min_lr=1e-7,
verbose=1),
        EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ]

    hist = self.model.fit(x_train, y_train, epochs=epochs, verbose=1,
validation_data=(x_val, y_val), callbacks=callbacks)
    return hist
```

Ví dụ: Sử dụng mạng ResNet50

```
# Load mô hình từ file
def load(self, model_file):
    self.model = tf.keras.models.load_model(model_file)

# Lưu mô hình vào file
def save(self, model_file):
    self.model.save(model_file)

# Tóm tắt kiến trúc mạng
def summary(self):
    self.model.summary()

# Thử nghiệm mô hình với dữ liệu ảnh đầu vào
def predict(self, x_test):
    return self.model.predict(x_test)
```



Ví dụ: Sử dụng mạng ResNet50

```
model = MyNetwork()  
model.build()  
model.summary()
```



Tài liệu tham khảo

- <https://arxiv.org/pdf/1704.04503>
- <https://arxiv.org/pdf/1705.02950>
- <https://learnopencv.com/classification-with-localization/>
- [Intuition and Implementation of Non-Max Suppression Algorithm in Object Detection | Towards Data Science](#)
- <https://johfischer.com/2021/10/30/simple-object-localization-and-classification-using-a-convolutional-network-with-tensorflow-keras/>